

Sales Visit Planner - Development

Name: Luke Doyle

Student ID: D00255656

1. System Overview
2. Architecture
3. Distributed Data Patterns
 - 3.1 Saga Pattern - Day Plan
 - 3.2 Transactional Outbox Pattern
 - 3.3 CQRS (Command Query Responsibility Segregation)
Expense Service CQRS Diagram:
4. Failure Handling
5. API Description
 - Customer and User Service
 - Planning Service
 - Expense Service
 - Worked Example: Saga Flow
6. Testing
7. Proof of Functionality
 - Set Up:
 - Planning:
 - Expenses:
 - Approving:
 - Denial & Re-Submit:
 - CQRS:
8. Reflection & Future Plans
9. Use of AI

1. System Overview

During my internship, I learned that Field-based sales representatives currently manage their working day across a variety of different disconnected tools. There is no single centralised system for planning visits, scheduling calls, recording outcomes, or tracking expenses. The result is inefficient routing, poor visibility for management, no reliable record of daily activity, and additional labour (as in less working with customers and more with business administration tasks).

The Sales Visit Planner aims to resolve this issue by providing a centralised system for Sales Representatives to use.

Three actors operate at the system boundary: the Sales Representative, the Manager, and the Administrator.

- Sales Representatives (reps) can manage their customers with planned visits and calls, log said visit and call outcomes, submit expenses for approval, and amend active plans.
- Managers can oversee teams of representatives' and individual representative's activity, flag priority customers, and approve or reject expense submissions.
- Administrators handle User (Manager and Sales Rep) account management across the platform.

2. Architecture

The system is composed of 4 application services, an Nginx Gateway, Apache Kafka, along with both Redis, and PostgreSQL databases. All components run as Docker containers.

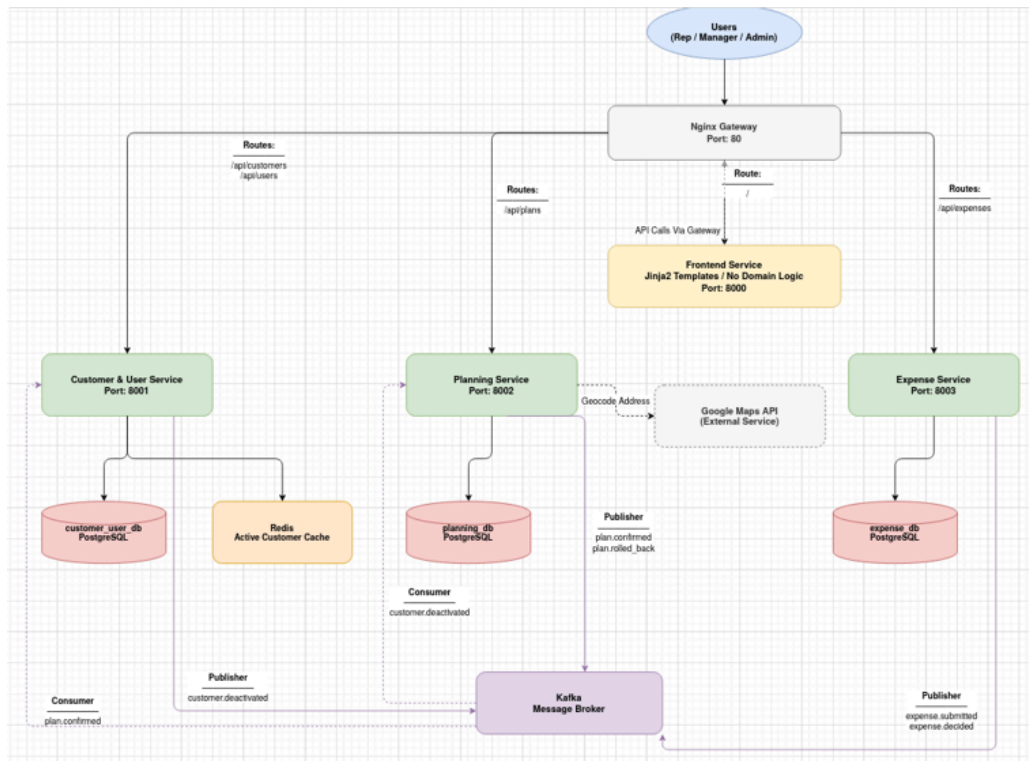
An Nginx Gateway acts as a single point of entry for all external traffic on port 80. It strips the `/api` prefix from incoming requests and routes by path to the appropriate backend service(s). Nothing is reachable directly.

Each service has its own PostgreSQL database. No service can query another service's database directly for either read or write. This helps keep the system secure and scalable. If a service needs data from another service's database, it either queries the API through the gateway or picks up a Kafka event. This keeps the services loosely coupled and allows each service's schema to evolve independently, following the core idea of Microservice architecture.

The four services are as follows.

- The Customer and User Service handles customer records, user accounts, and the active customer list. The list gets cached in Redis, this reduces database queries. This service is also where customer deactivation happens, it publishes a ``customer.deactivated`` event to Kafka when a customer is deactivated.
- The Planning Service owns the full “day plan” lifecycle. It runs saga orchestration, manages the transactional outbox, calls the Google Maps API, and maintains CQRS read projections. Confirming a plan writes the status change and the outbox event in one transaction.
- The Expense Service handles submission, approval, and rejection of expense claims. It publishes Kafka events when decisions are made and keeps a CQRS read model that managers query for expense history.
- The Frontend Service just renders Jinja2 templates. No domain logic, no database access. Everything it displays comes from the backend services via Nginx.

Kafka connects the Customer and User Service and the Planning Service. The two never call each other directly. All coordination goes through `plan.confirmed`, `plan.rolled_back`, and `customer.deactivated` topics.



3. Distributed Data Patterns

3.1 Saga Pattern - Day Plan

The Saga pattern handles transactions that touch more than one service. Instead of one big transaction, it breaks the work into local steps. If something fails midway, compensating transactions roll back to a prior state.

Two sagas are implemented here.

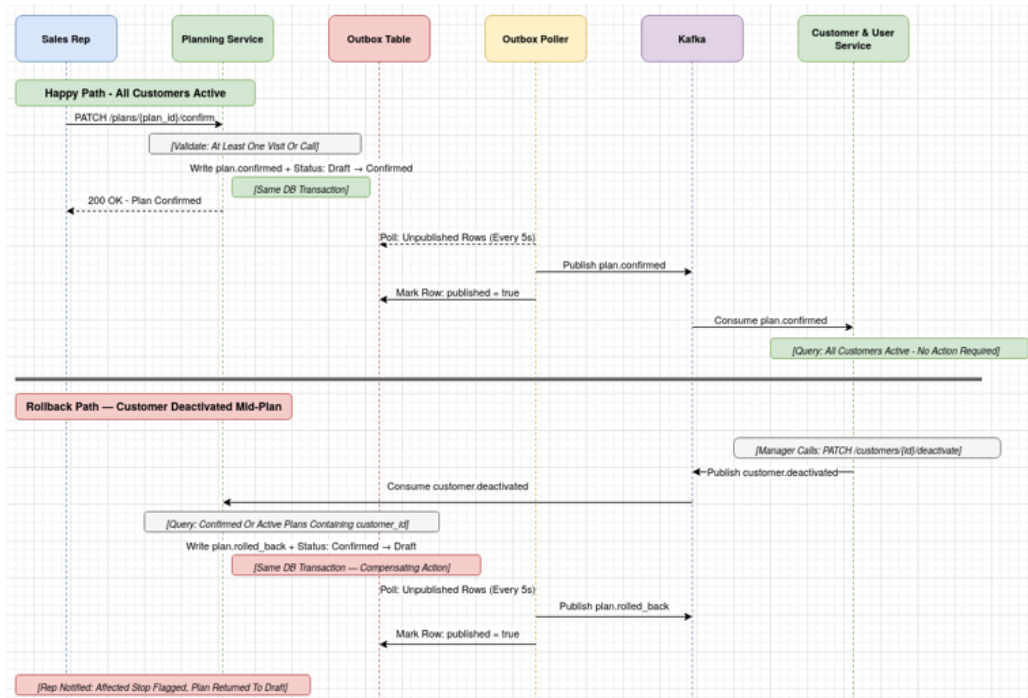
Plan Confirmation Saga

When a rep confirms a plan, the Planning Service moves it from `draft` to `confirmed` and writes a `plan.confirmed` event to the outbox, all in one database transaction. The outbox poller picks it up and publishes to Kafka. The Customer and User Service consumes it, checks every customer ID in the payload, and verifies they are all still active. If one has been deactivated, it publishes `plan.rolled_back` and the plan goes back to `draft`.

Customer Deactivation Saga

When a manager deactivates a customer via `PATCH /api/customers/{id}/deactivate`, the Customer and User Service sets `is_active = False`, clears the Redis cache, and fires a `customer.deactivated` event to Kafka. The Planning Service picks this up, finds every plan that has that customer on it in `confirmed` or `active` status, and rolls each one back to `draft`. Each rollback writes a `plan.rolled_back` outbox row in the same transaction. The poller publishes those out to Kafka.

Same compensating action in both cases. Plan goes back to `draft`, rep decides what to do next.



3.2 Transactional Outbox Pattern

The Transactional Outbox pattern aims to solve the “dual-write problem”. A service writes to the database, then tries to publish to Kafka. The database write succeeds. The Kafka publish fails. Services further downstream never get the event and the system is now inconsistent with no way to recover. This can become a huge problem.

With the outbox pattern, instead of publishing directly to Kafka, the Planning Service writes the event to an outbox table in the same database transaction as the state change. Both succeed or both fail. The event cannot be lost.

The pattern is clearest in the plan confirmation flow:

```

1 plan.status = DayPlanStatus.confirmed
2 outbox_row = Outbox(
3     event_type="plan.confirmed",
4     aggregate_id=plan.id,
5     payload={"plan_id": str(plan.id), "rep_id": str(plan.rep_id)},

```

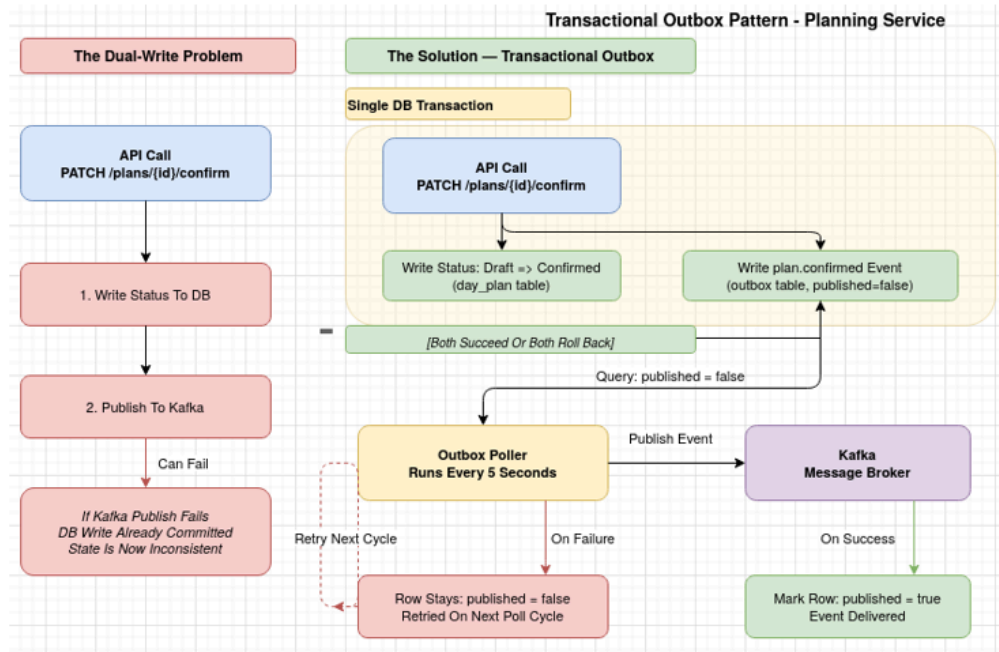
```

6 )
7 db.add(outbox_row)
8 await db.commit()

```

One commit. Status change and outbox row land together or not at all.

A background task in `outbox/poller.py` checks every five seconds and queries for any rows where `published` is `False`. For each one it calls the Kafka publisher. Success sets `published = True` and commits. Failure logs the error and leaves the row alone so it gets retried next cycle. One bad row does not hold up the others.



3.3 CQRS (Command Query Responsibility Segregation)

Command Query Responsibility Segregation (CQRS) splits the write model from the read model. Writes handle state transitions. Reads query a separate projection table built for that specific query, without touching the write tables at all.

Both the Planning Service and the Expense Service use this.

In the Planning Service, every function that changes a plan calls `update_plan_projection()` straight after committing. It reads the current plan state, counts visits and calls, and upserts a row in `PlanReadModel`. The read endpoints in `routers/plans_read.py` only ever query `PlanReadModel`. `DayPlan`, `Visit`, and `Call` are never touched from the read side.

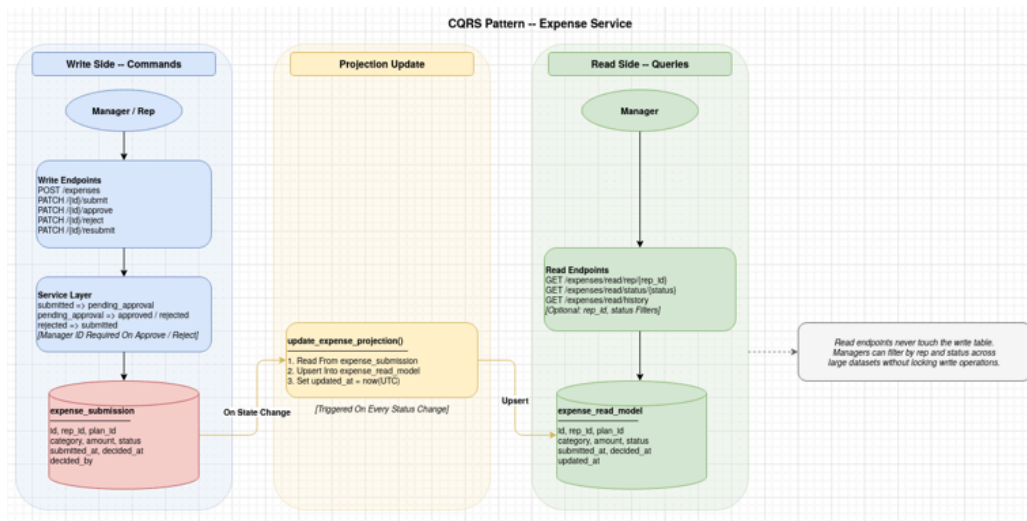
Same approach in the Expense Service. Every state change calls `update_expense_projection()` after committing, which upserts into

`ExpenseReadModel` with the current status, amount, category, and decision metadata. Read endpoints in `routers/expenses_read.py` query `ExpenseReadModel` only.

The reason this is worth doing for the expense view specifically is that a manager filtering expenses by rep and status across a large dataset is purely a read operation. Keeping that off the write table means it never competes with or blocks an ongoing state transition.

Because the projection updates in the same request as the write, there is no lag. Read model state is accurate immediately.

Expense Service CQRS Diagram:



4. Failure Handling

Four failure scenarios are handled in the system.

External Mapping Service Unavailable

If the Google Maps API is unreachable when the Planning Service needs coordinates for a customer, the plan stays in `draft`. The rep is notified and nothing proceeds silently. Either the service comes back, the rep removes that customer, or they provide coordinates manually. Coordinates are cached against the customer record after the first successful call, so this gets less likely over time as more customers build up stored coordinates.

Customer Deactivated Mid-Plan

Covered by the customer deactivation saga. The Customer and User Service publishes `customer.deactivated` to Kafka, the Planning Service finds every affected plan, rolls each back to `draft` in a single transaction. The rep is notified and keeps control of what happens next. The plan is not cancelled automatically.

Outbox Publish Failure

If Kafka is unavailable when the poller runs, the row stays unpublished and gets retried on the next cycle five seconds later. Failures are tracked per row so one problem does not block anything else. As long as the database is up, no event is lost.

Kafka Consumer Startup

On boot, consumers can hit `GroupCoordinatorNotAvailable` errors while Kafka is still coming up. This is expected. Consumers retry automatically and settle within a few seconds once the broker is ready. No manual intervention needed.

5. API Description

All paths are as seen through Nginx. The `/api` prefix is stripped before the request reaches the service. A full endpoint list is in the README.

The following tables are taken from the README.

Customer and User Service

Metho d	Path	Description
GET	<code>/api/customers</code>	List all active customers, served from Redis cache when available
POST	<code>/api/customers</code>	Create a new customer record
PATC H	<code>/api/customers/{id}</code> <code>/deactivate</code>	Soft-delete a customer, publishes <code>customer.deactivated</code> to Kafka
PATC H	<code>/api/customers/{id}</code> <code>/flag</code>	Set the priority flag, requires <code>X-User-Id</code> of a manager
POST	<code>/api/users</code>	Create a new user account, password is bcrypt-hashed
DELET E	<code>/api/users/{id}</code>	Hard-delete a user, requires <code>X-User-Id</code> of an administrator

Planning Service

Method	Path	Description
POST	/api/plans	Create a day plan in <code>draft</code> state
PATCH	/api/plans/{id}/confirm	Transition <code>draft</code> → <code>confirmed</code> , requires at least one visit or call
PATCH	/api/plans/{id}/activate	Transition <code>confirmed</code> → <code>active</code>
POST	/api/plans/{id}/visits	Add a customer visit to a plan
PATCH	/api/plans/{id}/visits/{vid}	Update visit status and outcome notes
GET	/api/plans/read/history	Read model: full plan history, accepts <code>?rep_id</code> and <code>?date</code>

Expense Service

Method	Path	Description
POST	/api/expenses	Create an expense in <code>submitted</code> state
PATCH	/api/expenses/{id}/approve	Transition to <code>approved</code> , requires <code>manager_id</code> in body
PATCH	/api/expenses/{id}/reject	Transition to <code>rejected</code> , requires <code>manager_id</code> in body
PATCH	/api/expenses/{id}/resubmit	Transition <code>rejected</code> → <code>submitted</code> , optionally amend details
GET	/api/expenses/read/history	Read model: full expense history, accepts <code>?rep_id</code> and <code>?status</code>

Worked Example: Saga Flow

Create a plan, add a visit, then confirm it.


```

1 curl -X POST http://localhost/api/plans \
2   -H "Content-Type: application/json" \
3   -d '{"rep_id": "abc-123", "date": "2026-04-09"}'
4
5 curl -X POST http://localhost/api/plans/{plan_id}/visits \
6   -H "Content-Type: application/json" \
7   -d '{"customer_id": "cust-456", "customer_name": "Acme Ltd",
8     "scheduled_order": 1}'
9
10 curl -X PATCH http://localhost/api/plans/{plan_id}/confirm

```

Confirming the plan kicks off the saga. The Planning Service writes the status change and the outbox event together in one transaction. The poller picks it up and publishes to Kafka. The Customer and User Service checks every customer on the plan. All active, plan stays confirmed. Any deactivated, it publishes `plan.rolled_back` and the plan drops back to `draft`.

6. Testing

The Test suite is in `tests/` at the project root. Tests send HTTP requests to the running Docker stack through Nginx, so the stack must be up before running them.

Install the test dependencies:

```

1 python -m venv .venv
2 source .venv/bin/activate
3 pip install pytest httpx pytest-asyncio

```

Run all tests:

```

1 pytest tests/

```

Five test files: `test_plans.py`, `test_expenses.py`, `test_cqrs.py`, `test_saga.py`, and `test_edge_cases.py`. Each is self-contained and pulls from function-scoped fixtures in `tests/conftest.py`. Fixtures create their own users, customers, and plans over HTTP so nothing depends on existing database state.

Saga tests poll the plan endpoint after triggering a Kafka-driven rollback, up to ten retries at 0.5 second intervals. This is needed because the rollback goes through the Kafka consumer and the outbox poller, both async, so the final state takes a moment to land.

7. Proof of Functionality

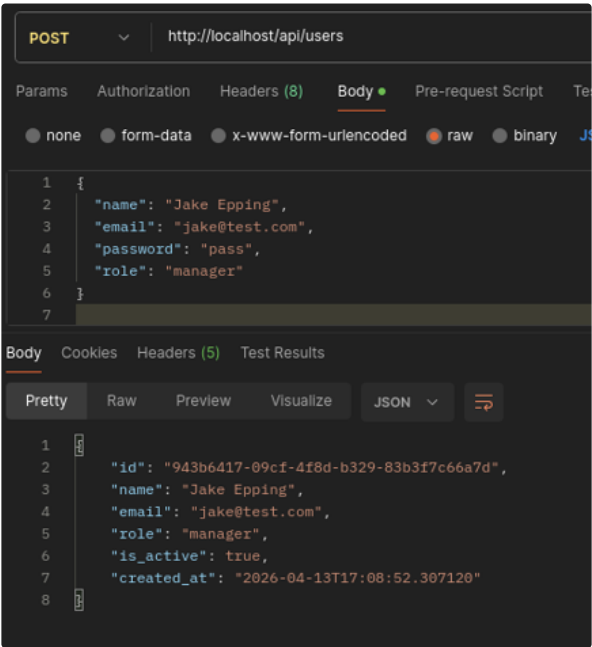
The following is screenshots taken by myself of the system fully functional.

Each screenshot will show a postman request in JSON to a specific URL along with a screenshot from the Docker logs.

the names and locations used are fake (I had a little bit of fun).

Set Up:

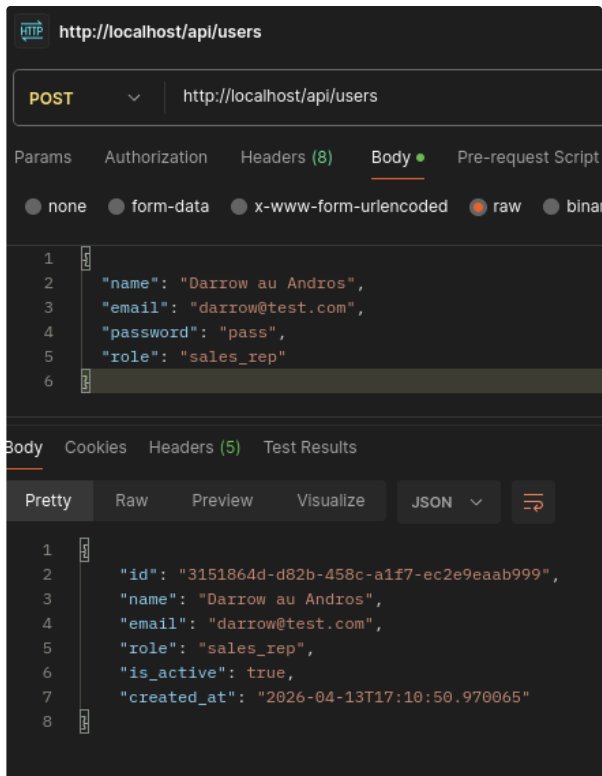
Manager:



Customer & User Service Log:

```
2026-04-13 17:01:55 [INFO] plan.confirmed consumer started (group=customer_user_service, topic=plan.confirmed).
2026-04-13 17:08:52 [DEBUG] Database session opened.
2026-04-13 17:08:52 [DEBUG] User created: 943b6417-09cf-4f8d-b329-83b3f7c66a7d (UserRole.manager).
2026-04-13 17:08:52 [DEBUG] Database session closed.
INFO:      172.18.0.12:42120 - "POST /users HTTP/1.0" 201 Created
```

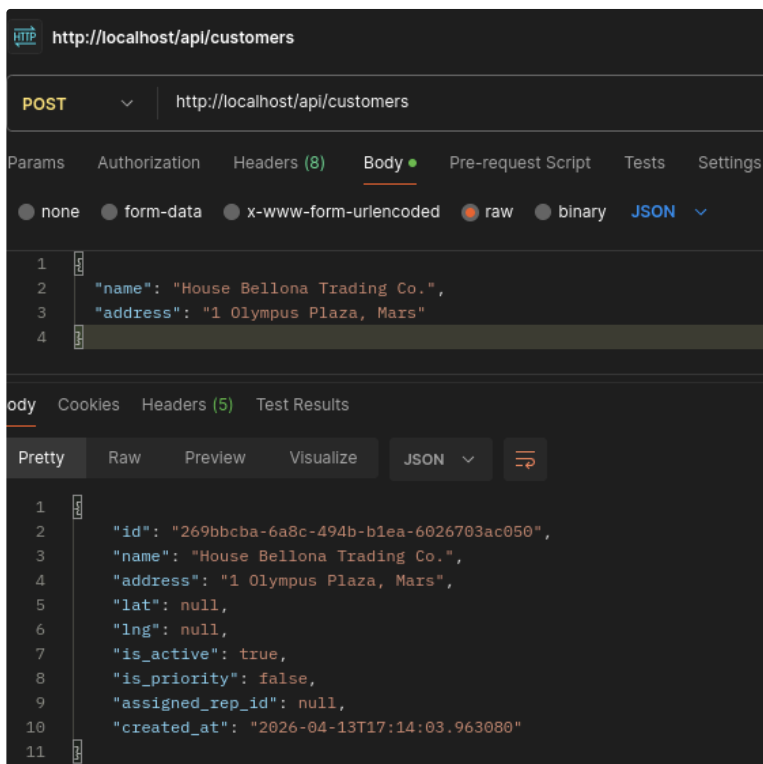
Sales Rep:



Customer & User Service Log:

```
2026-04-13 17:10:50 [DEBUG] Database session opened.
2026-04-13 17:10:50 [DEBUG] User created: 3151864d-d82b-458c-a1f7-ec2e9eaab999 (UserRole.sales_rep).
2026-04-13 17:10:50 [DEBUG] Database session closed.
INFO:      172.18.0.12:36238 - "POST /users HTTP/1.0" 201 Created
```

Customer:



Customer & User Service Log:

```
2026-04-13 17:14:03 [DEBUG] Database session opened.
2026-04-13 17:14:03 [DEBUG] Active customers cache invalidated.
2026-04-13 17:14:03 [DEBUG] Customer created: 269bbcba-6a8c-494b-b1ea-6026703ac050.
2026-04-13 17:14:03 [DEBUG] Database session closed.
INFO:      172.18.0.12:35980 - "POST /customers HTTP/1.0" 201 Created
```

Additional Customers for testing, using `curl` to make testing quicker. Cleaned Output Below (AI Fixed)

```
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$ curl -X POST http://localhost/api/customers \
-H "Content-Type: application/json" \
-d '{"name": "Reaper Logistics", "address": "Institute Grounds, Luna"}'
{"id": "80c0282b-c2be-4b34-8423-936e834a82a3", "name": "Reaper Logistics", "address": "Institute Grounds, Luna", "lat": null, "lng": null, "is_active": true, "is_priority": false, "assigned_rep_id": null, "created_at": "2026-04-13T17:16:22.423431"} (.venv) d
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$ curl -X POST http://localhost/api/customers \
-H "Content-Type: application/json" \
-d '{"name": "Morcerf & Associates", "address": "Rue du Helder, Paris"}'
{"id": "37f50ec8-4687-41af-a50b-42d8676d6d7f", "name": "Morcerf & Associates", "address": "Rue du Helder, Paris", "lat": null, "lng": null, "is_active": true, "is_priority": false, "assigned_rep_id": null, "created_at": "2026-04-13T17:16:28.778612"} (.venv) d
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$ curl -X POST http://localhost/api/customers \
-H "Content-Type: application/json" \
-d '{"name": "Gestral Expeditions", "address": "The Forgotten Shore, Lumiere"}'
{"id": "d118a529-0c7d-438a-8bd4-5d81d4f48828", "name": "Gestral Expeditions", "address": "The Forgotten Shore, Lumiere", "lat": null, "lng": null, "is_active": true, "is_priority": false, "assigned_rep_id": null, "created_at": "2026-04-13T17:16:33.527839"} (.venv) d
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$ curl -X POST http://localhost/api/customers \
-H "Content-Type: application/json" \
-d '{"name": "Nilfgaard Merchant Guild", "address": "Vizima Trade Quarter, Temeria"}'
{"id": "fe9bfa9f-287f-4633-a990-133f9254f254", "name": "Nilfgaard Merchant Guild", "address": "Vizima Trade Quarter, Temeria", "lat": null, "lng": null, "is_active": true, "is_priority": false, "assigned_rep_id": null, "created_at": "2026-04-13T17:16:37.915229"} (.venv) d
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$
```

```
1 POST /api/customers → 201 Created
2
3 {
4   "id": "80c0282b-c2be-4b34-8423-936e834a82a3",
5   "name": "Reaper Logistics",
6   "address": "Institute Grounds, Luna",
7   "lat": null,
8   "lng": null,
9   "is_active": true,
10  "is_priority": false,
11  "assigned_rep_id": null,
12  "created_at": "2026-04-13T17:16:22.423431"
13 }
14
15 ---
16
17 POST /api/customers → 201 Created
18
19 {
20   "id": "37f50ec8-4687-41af-a50b-42d8676d6d7f",
21   "name": "Morcerf & Associates",
22   "address": "Rue du Helder, Paris",
23   "lat": null,
24   "lng": null,
25   "is_active": true,
26   "is_priority": false,
27   "assigned_rep_id": null,
28   "created_at": "2026-04-13T17:16:28.778612"
29 }
30
31 ---
32
33 POST /api/customers → 201 Created
34
```

```

35 {
36   "id": "d118a529-0c7d-438a-8bd4-5d81d4f48828",
37   "name": "Gestral Expeditions",
38   "address": "The Forgotten Shore, Lumiere",
39   "lat": null,
40   "lng": null,
41   "is_active": true,
42   "is_priority": false,
43   "assigned_rep_id": null,
44   "created_at": "2026-04-13T17:16:33.527839"
45 }
46
47 ---
48
49 POST /api/customers → 201 Created
50
51 {
52   "id": "fe9bfa9f-287f-4633-a990-133f9254f254",
53   "name": "Nilfgaard Merchant Guild",
54   "address": "Vizima Trade Quarter, Temeria",
55   "lat": null,
56   "lng": null,
57   "is_active": true,
58   "is_priority": false,
59   "assigned_rep_id": null,
60   "created_at": "2026-04-13T17:16:37.915254"
61 }
62

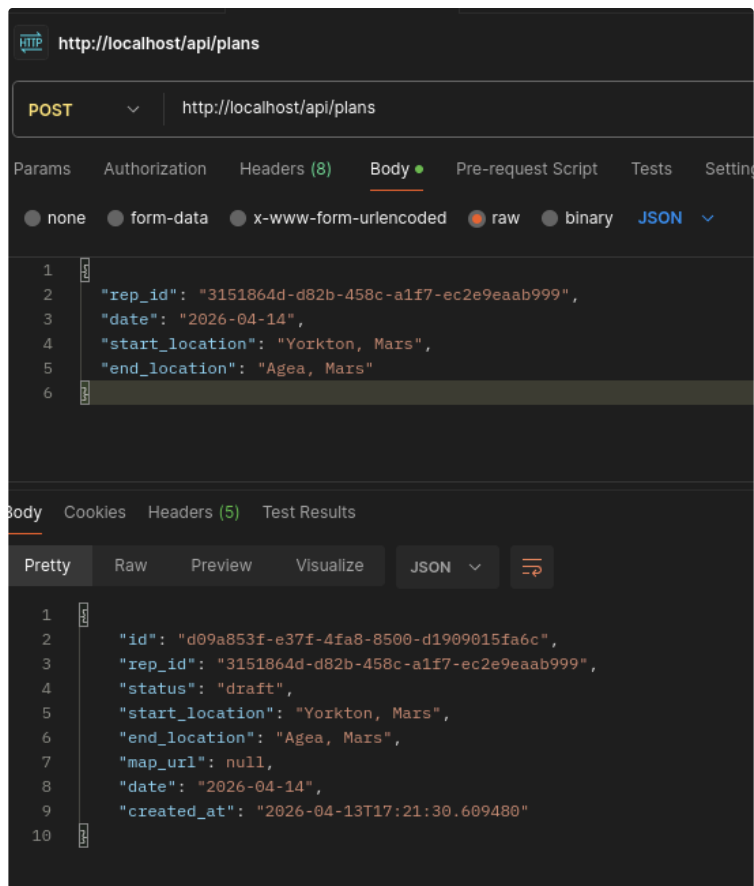
```

Planning:

Creating a Plan:

A Plan is returned with a URL.

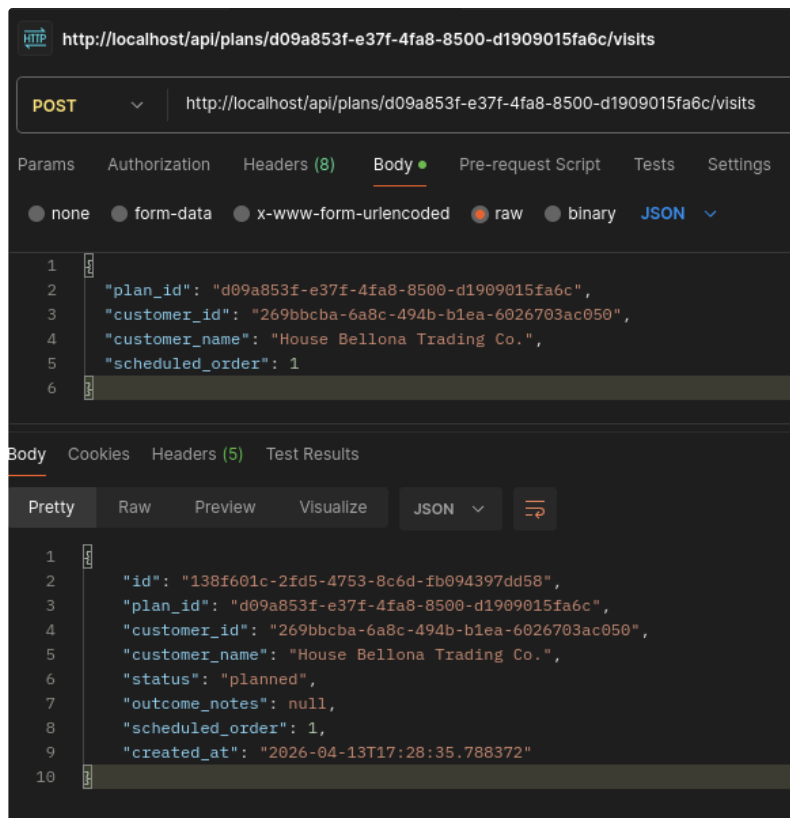
There is no URL in this example due to missing API Key, instead we have a default `null` value.



Planning Service Log:

```
2026-04-13 17:21:30 [DEBUG] Database session opened.
2026-04-13 17:21:30 [DEBUG] Projection inserted for plan d09a853f-e37f-4fa8-8500-d1909015fa6c.
2026-04-13 17:21:30 [INFO] Plan projection upserted for plan d09a853f-e37f-4fa8-8500-d1909015fa6c (0 visits, 0 calls).
2026-04-13 17:21:30 [INFO] Plan created: d09a853f-e37f-4fa8-8500-d1909015fa6c.
2026-04-13 17:21:30 [DEBUG] POST /plans -- created plan d09a853f-e37f-4fa8-8500-d1909015fa6c.
2026-04-13 17:21:30 [DEBUG] Database session closed.
INFO: 172.18.0.12:35796 - "POST /plans HTTP/1.0" 201 Created
```

Adding Customers to a Plan:



Planning Service Log:

```
2026-04-13 17:28:35 [DEBUG] Database session opened.
2026-04-13 17:28:35 [DEBUG] Projection updated for plan d09a853f-e37f-4fa8-8500-d1909015fa6c.
2026-04-13 17:28:35 [INFO] Plan projection upserted for plan d09a853f-e37f-4fa8-8500-d1909015fa6c (1 visits, 0 calls).
2026-04-13 17:28:35 [DEBUG] Visit added to plan d09a853f-e37f-4fa8-8500-d1909015fa6c: 138f601c-2fd5-4753-8c6d-fb094397dd58.
2026-04-13 17:28:35 [DEBUG] POST /plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits → created visit 138f601c-2fd5-4753-8c6d-fb094397dd58.
2026-04-13 17:28:35 [DEBUG] Database session closed.
INFO: 172.18.0.12:52124 - "POST /plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits HTTP/1.0" 201 Created
```

```
(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$ curl -X POST http://localhost/api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits \
-H "Content-Type: application/json" \
-d '{"plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c", "customer_id": "80c0282b-c2be-4b34-8423-936e834a82a3", "customer_name": "Reaper Logistics", "scheduled_order": 2}'
{"id": "e1bd8832-838b-4d1b-9d24-0234e4b4cc9c", "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c", "customer_id": "80c0282b-c2be-4b34-8423-936e834a82a3", "customer_name": "Reaper Logistics", "status": "planned", "outcome_notes": null, "scheduled_order": 2, "created_at": "2026-04-13T17:30:15.177636"}(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$
(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$ curl -X POST http://localhost/api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits \
-H "Content-Type: application/json" \
-d '{"plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c", "customer_id": "37f50ec8-4687-41af-a50b-42d8676d6d7f", "customer_name": "Morcerf & Associates", "scheduled_order": 3}'
{"id": "4a007700-ddf8-4896-acb1-fa0e4e93cd04", "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c", "customer_id": "37f50ec8-4687-41af-a50b-42d8676d6d7f", "customer_name": "Morcerf & Associates", "status": "planned", "outcome_notes": null, "scheduled_order": 3, "created_at": "2026-04-13T17:30:19.998977"}(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$
(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$ curl -X POST http://localhost/api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits \
-H "Content-Type: application/json" \
-d '{"plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c", "customer_id": "d118a529-0c7d-438a-8bd4-5d81d4f48828", "customer_name": "Gestral Expeditions", "scheduled_order": 4}'
{"id": "406c9e2c-3867-4014-88a0-7cdbab226b59", "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c", "customer_id": "d118a529-0c7d-438a-8bd4-5d81d4f48828", "customer_name": "Gestral Expeditions", "status": "planned", "outcome_notes": null, "scheduled_order": 4, "created_at": "2026-04-13T17:30:24.473460"}(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$
(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$ curl -X POST http://localhost/api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits \
-H "Content-Type: application/json" \
-d '{"plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c", "customer_id": "fe9bfa9f-287f-4633-a990-133f9254f254", "customer_name": "Nilfgaard Merchant Guild", "scheduled_order": 5}'
{"id": "8306a0dd-c5be-46a1-a66f-f7a422babfdb", "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c", "customer_id": "fe9bfa9f-287f-4633-a990-133f9254f254", "customer_name": "Nilfgaard Merchant Guild", "status": "planned", "outcome_notes": null, "scheduled_order": 5, "created_at": "2026-04-13T17:30:30.021195"}(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$
(.venv) duck@duckpad:~/Documents/college/year4/microservices/cal/svp_mic$
```

```
1 POST /api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits → 201 Created
```

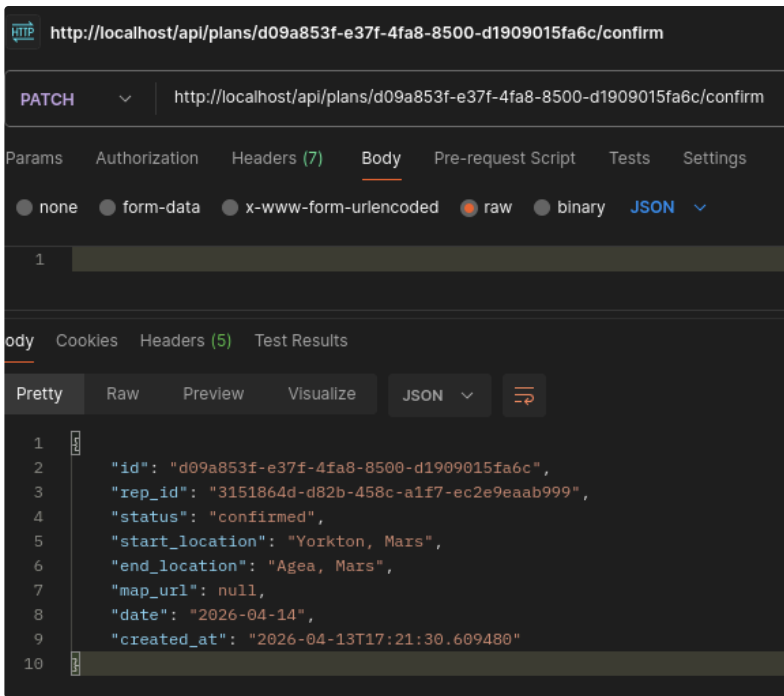


```

2
3 {
4   "id": "e1bd8832-838b-4d1b-9d24-0234e4b4cc9c",
5   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
6   "customer_id": "80c0282b-c2be-4b34-8423-936e834a82a3",
7   "customer_name": "Reaper Logistics",
8   "status": "planned",
9   "outcome_notes": null,
10  "scheduled_order": 2,
11  "created_at": "2026-04-13T17:30:15.177636"
12 }
13
14 ---
15
16 POST /api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits → 201
17 Created
18 {
19   "id": "4a007700-ddf8-4896-acb1-fa0e4e93cd04",
20   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
21   "customer_id": "37f50ec8-4687-41af-a50b-42d8676d6d7f",
22   "customer_name": "Morcerf & Associates",
23   "status": "planned",
24   "outcome_notes": null,
25   "scheduled_order": 3,
26   "created_at": "2026-04-13T17:30:19.998977"
27 }
28
29 ---
30
31 POST /api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits → 201
32 Created
33 {
34   "id": "406c9e2c-3867-4014-88a0-7cdbab226b59",
35   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
36   "customer_id": "d118a529-0c7d-438a-8bd4-5d81d4f48828",
37   "customer_name": "Gestral Expeditions",
38   "status": "planned",
39   "outcome_notes": null,
40   "scheduled_order": 4,
41   "created_at": "2026-04-13T17:30:24.473460"
42 }
43
44 ---
45
46 POST /api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/visits → 201
47 Created
48 {
49   "id": "8306a0dd-c5be-46a1-a66f-f7a422babfdb",
50   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
51   "customer_id": "fe9bfa9f-287f-4633-a990-133f9254f254",
52   "customer_name": "Nilfgaard Merchant Guild",
53   "status": "planned",
54   "outcome_notes": null,
55   "scheduled_order": 5,
56   "created_at": "2026-04-13T17:30:30.021195"
57 }
58

```

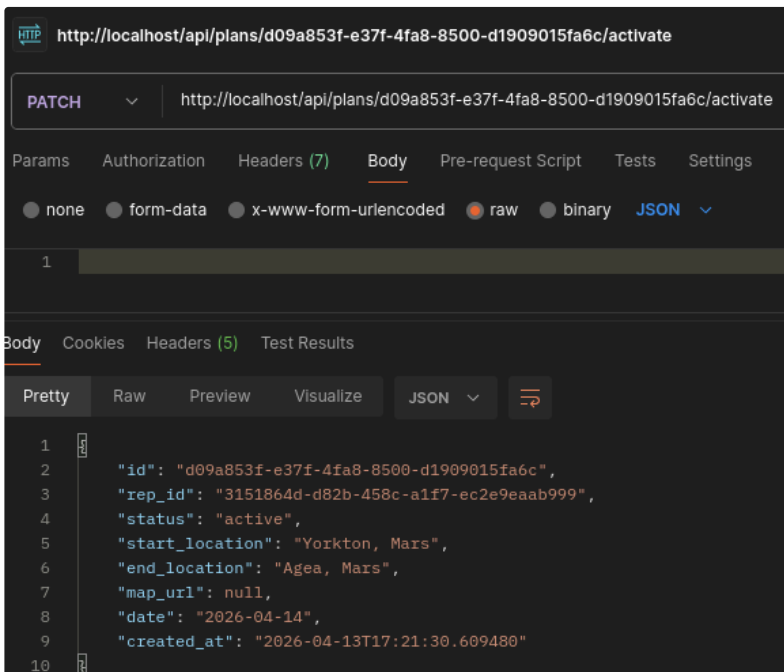
Confirm Plan:



Planning Service Log:

```
2026-04-13 17:33:23 [DEBUG] Database session opened.
2026-04-13 17:33:23 [DEBUG] Projection updated for plan d09a853f-e37f-4fa8-8500-d1909015fa6c.
2026-04-13 17:33:23 [INFO] Plan projection upserted for plan d09a853f-e37f-4fa8-8500-d1909015fa6c (5 visits, 0 calls).
2026-04-13 17:33:23 [INFO] Plan confirmed: d09a853f-e37f-4fa8-8500-d1909015fa6c.
2026-04-13 17:33:23 [DEBUG] Database session closed.
INFO: 172.18.0.12:56454 - "PATCH /plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/confirm HTTP/1.0" 200 OK
2026-04-13 17:33:27 [DEBUG] Outbox poller: found 1 pending row(s).
2026-04-13 17:33:27 [INFO] Published event 'plan.confirmed' to topic 'plan.confirmed'.
2026-04-13 17:33:27 [INFO] Outbox row cb005f08-bb30-4b88-b907-550344569f04 marked published (event_type='plan.confirmed').
2026-04-13 17:33:32 [DEBUG] Outbox poller: no pending rows.
```

Activate a Plan:



Planning Service Log:

```
2026-04-13 17:35:15 [DEBUG] Database session opened.
2026-04-13 17:35:15 [DEBUG] Projection updated for plan d09a853f-e37f-4fa8-8500-d1909015fa6c.
2026-04-13 17:35:15 [INFO] Plan projection upserted for plan d09a853f-e37f-4fa8-8500-d1909015fa6c (5 visits, 0 calls).
2026-04-13 17:35:15 [INFO] Plan activated: d09a853f-e37f-4fa8-8500-d1909015fa6c.
INFO:      172.18.0.12:40414 - "PATCH /plans/d09a853f-e37f-4fa8-8500-d1909015fa6c/activate HTTP/1.0" 200 OK
2026-04-13 17:35:15 [DEBUG] Database session closed.
```

Saga - De-activate Customer:

Using a Patch Request, a Manager can deactivate a customer during a planned route.

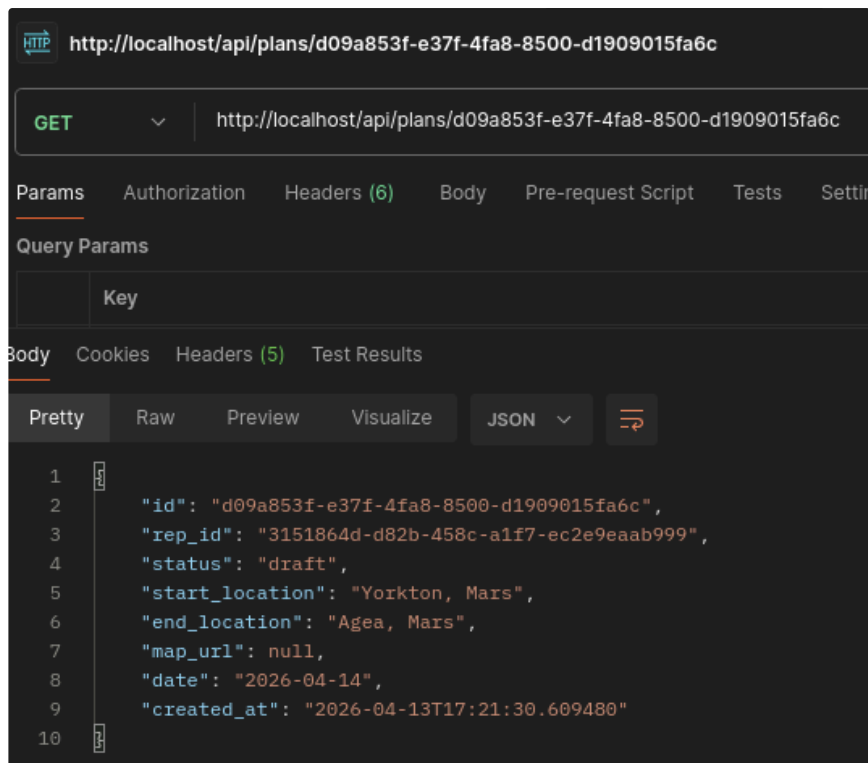
The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost/api/customers/fe9bfa9f-287f-4633-a990-133f9254f254/deactivate`
- Method:** PATCH
- Body:** The request body is in JSON format, containing the following fields:

```
{
  "id": "fe9bfa9f-287f-4633-a990-133f9254f254",
  "name": "Nilfgaard Merchant Guild",
  "address": "Vizima Trade Quarter, Temeria",
  "lat": null,
  "lng": null,
  "is_active": false,
  "is_priority": false,
  "assigned_rep_id": null,
  "created_at": "2026-04-13T17:16:37.915229"
}
```

```
2026-04-13 17:38:47 [DEBUG] Received customer.deactivated message at offset 0.
2026-04-13 17:38:47 [DEBUG] Queued rollback for plan d09a853f-e37f-4fa8-8500-d1909015fa6c (customer fe9bfa9f-287f-4633-a990-133f9254f254).
2026-04-13 17:38:47 [INFO] Rolled back 1 plan(s) for deactivated customer fe9bfa9f-287f-4633-a990-133f9254f254.
2026-04-13 17:38:52 [DEBUG] Outbox poller: found 1 pending row(s).
Topic plan.rolled_back is not available during auto-create initialization
2026-04-13 17:38:52 [INFO] Published event 'plan.rolled_back' to topic 'plan.rolled_back'.
2026-04-13 17:38:52 [INFO] Outbox row 855e89f7-a89f-444f-baab-90483159e0de marked published (event_type='plan.rolled_back').
```

Now the Plan is back in Drafts:



```
HTTP http://localhost/api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c

GET http://localhost/api/plans/d09a853f-e37f-4fa8-8500-d1909015fa6c

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

Key

Body Cookies Headers (5) Test Results

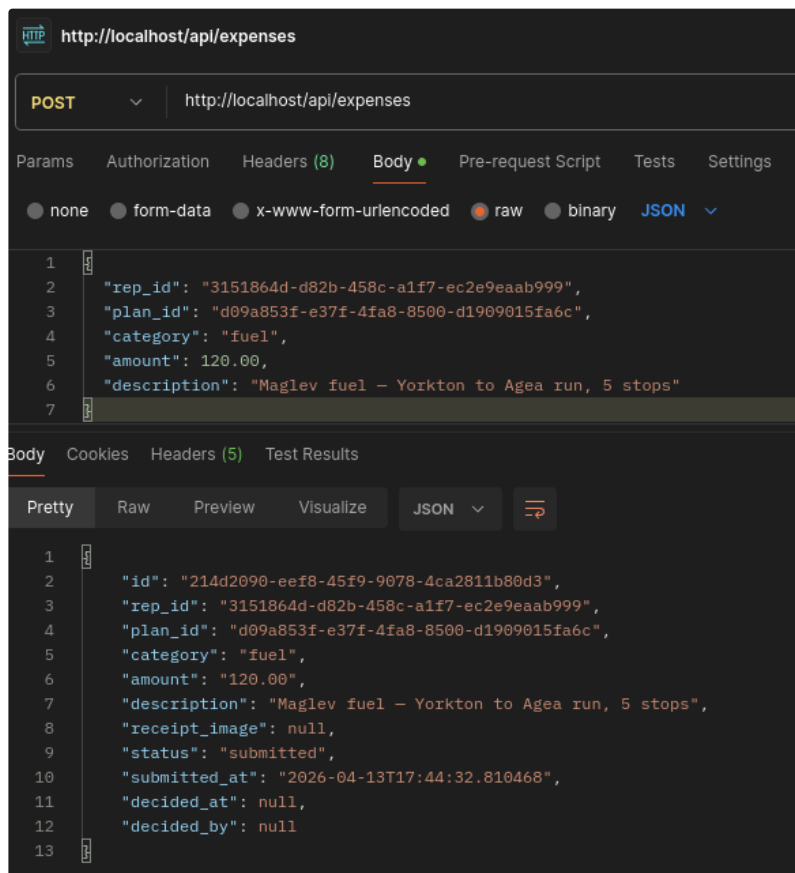
Pretty Raw Preview Visualize JSON ↗

1 {
2   "id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
3   "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
4   "status": "draft",
5   "start_location": "Yorkton, Mars",
6   "end_location": "Agea, Mars",
7   "map_url": null,
8   "date": "2026-04-14",
9   "created_at": "2026-04-13T17:21:30.609480"
10 }
```

Expenses:

Approving:

Expense Created:



```
HTTP http://localhost/api/expenses

POST http://localhost/api/expenses

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON ↗

1 {
2   "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
3   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
4   "category": "fuel",
5   "amount": 120.00,
6   "description": "Maglev fuel - Yorkton to Agea run, 5 stops"
7 }

Body Cookies Headers (5) Test Results

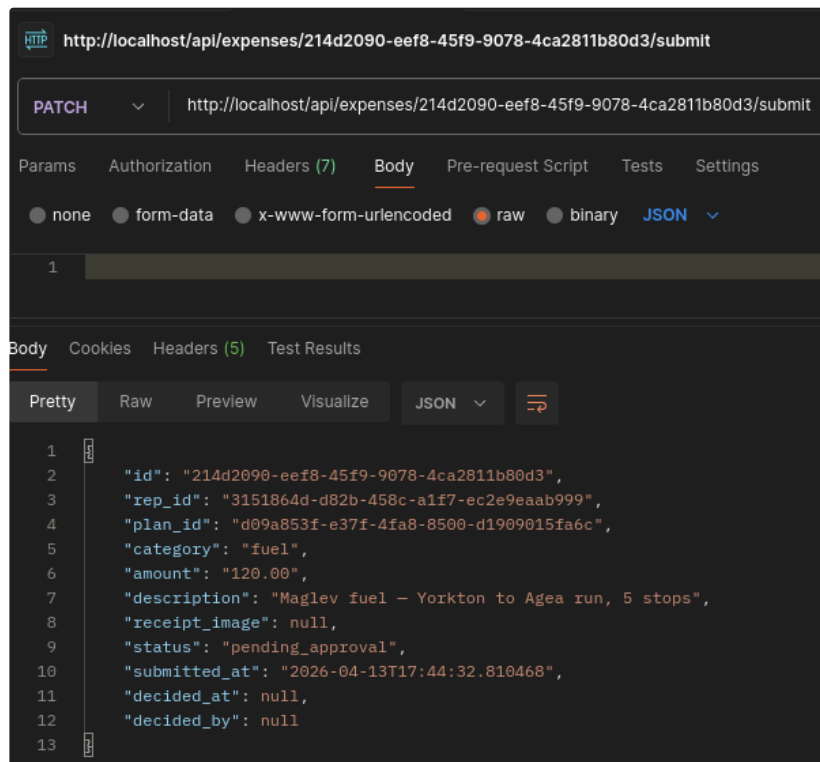
Pretty Raw Preview Visualize JSON ↗

1 {
2   "id": "214d2090-eef8-45f9-9078-4ca2811b80d3",
3   "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
4   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
5   "category": "fuel",
6   "amount": "120.00",
7   "description": "Maglev fuel - Yorkton to Agea run, 5 stops",
8   "receipt_image": null,
9   "status": "submitted",
10  "submitted_at": "2026-04-13T17:44:32.810468",
11  "decided_at": null,
12  "decided_by": null
13 }
```

Expense Service Log:

```
2026-04-13 17:01:55 [INFO] expense.decided consumer started (group=expense_service).
2026-04-13 17:44:32 [DEBUG] Database session opened.
2026-04-13 17:44:32 [DEBUG] Projection inserted for expense 214d2090-eef8-45f9-9078-4ca2811b80d3.
2026-04-13 17:44:32 [INFO] Expense projection upserted for expense 214d2090-eef8-45f9-9078-4ca2811b80d3.
2026-04-13 17:44:32 [INFO] Expense submission created: 214d2090-eef8-45f9-9078-4ca2811b80d3.
2026-04-13 17:44:32 [DEBUG] POST /expenses → created expense 214d2090-eef8-45f9-9078-4ca2811b80d3.
2026-04-13 17:44:32 [DEBUG] Database session closed.
```

Expense Submitted:



The screenshot shows a web browser interface for a REST client. The URL bar displays `http://localhost/api/expenses/214d2090-eef8-45f9-9078-4ca2811b80d3/submit`. The method is set to `PATCH`. The body tab is active, showing a JSON payload with the following fields:

```
{
  "id": "214d2090-eef8-45f9-9078-4ca2811b80d3",
  "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
  "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
  "category": "fuel",
  "amount": "120.00",
  "description": "Maglev fuel - Yorkton to Agea run, 5 stops",
  "receipt_image": null,
  "status": "pending_approval",
  "submitted_at": "2026-04-13T17:44:32.810468",
  "decided_at": null,
  "decided_by": null
}
```

Expense Service Log:

```
2026-04-13 17:48:39 [DEBUG] Database session opened.
2026-04-13 17:48:39 [DEBUG] Projection updated for expense 214d2090-eef8-45f9-9078-4ca2811b80d3.
2026-04-13 17:48:39 [INFO] Expense projection upserted for expense 214d2090-eef8-45f9-9078-4ca2811b80d3.
2026-04-13 17:48:39 [INFO] Expense 214d2090-eef8-45f9-9078-4ca2811b80d3 transitioned to pending_approval.
2026-04-13 17:48:39 [DEBUG] Database session closed.
INFO: 172.18.0.12:52732 - "PATCH /expenses/214d2090-eef8-45f9-9078-4ca2811b80d3/submit HTTP/1.0" 200 OK
```

Approve as Manager:

PATCH ▼ http://localhost/api/expenses/214d2090-eef8-45f9-9078-4ca2811b80d3/approve

Params Authorization Headers (8) **Body** ● Pre-request Script Tests Settings

● none ● form-data ● x-www-form-urlencoded ● raw ● binary JSON ▼

```
1
2 "manager_id": "943b6417-09cf-4f8d-b329-83b3f7c66a7d"
3
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON ▼

```
1
2 "id": "214d2090-eef8-45f9-9078-4ca2811b80d3",
3 "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
4 "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
5 "category": "fuel",
6 "amount": "120.00",
7 "description": "Maglev fuel - Yorkton to Agea run, 5 stops",
8 "receipt_image": null,
9 "status": "approved",
10 "submitted_at": "2026-04-13T17:44:32.810468",
11 "decided_at": "2026-04-13T17:51:12.499830",
12 "decided_by": "943b6417-09cf-4f8d-b329-83b3f7c66a7d"
13
```

Expense Service Log:

```
2026-04-13 17:51:12 [DEBUG] Database session opened.
2026-04-13 17:51:12 [DEBUG] Projection updated for expense 214d2090-eef8-45f9-9078-4ca2811b80d3.
2026-04-13 17:51:12 [INFO] Expense projection upserted for expense 214d2090-eef8-45f9-9078-4ca2811b80d3.
2026-04-13 17:51:12 [INFO] Published event 'expense.decided' to topic 'expense.decided'.
2026-04-13 17:51:12 [INFO] Expense 214d2090-eef8-45f9-9078-4ca2811b80d3 approved by manager 943b6417-09cf-4f8d-b329-83b3f7c66a7d.
2026-04-13 17:51:12 [DEBUG] Database session closed.
INFO: 172.18.0.12:53768 - "PATCH /expenses/214d2090-eef8-45f9-9078-4ca2811b80d3/approve HTTP/1.0" 200 OK
2026-04-13 17:51:12 [DEBUG] Received expense.decided message at offset 0.
2026-04-13 17:51:12 [INFO] expense.decided received: expense_id=214d2090-eef8-45f9-9078-4ca2811b80d3 decision=approved.
```

Denial & Re-Submit:

POST ▼ `http://localhost/api/expenses`

Params Authorization Headers (8) **Body** ● Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** ▼

```
1 {
2   "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
3   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
4   "category": "overnight_stay",
5   "amount": 85.00,
6   "description": "Overnight stay - Cat's Eye Inn, Novigrad"
7 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize **JSON** ▼

```
1 {
2   "id": "b029f4d8-211f-45e2-9fa4-e44826078fc4",
3   "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
4   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
5   "category": "overnight_stay",
6   "amount": "85.00",
7   "description": "Overnight stay - Cat's Eye Inn, Novigrad",
8   "receipt_image": null,
9   "status": "submitted",
10  "submitted_at": "2026-04-13T17:58:06.447979",
11  "decided_at": null,
12  "decided_by": null
13 }
```

PATCH ▼ `http://localhost/api/expenses/b029f4d8-211f-45e2-9fa4-e44826078fc4/submit`

Params Authorization Headers (8) **Body** ● Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** ▼

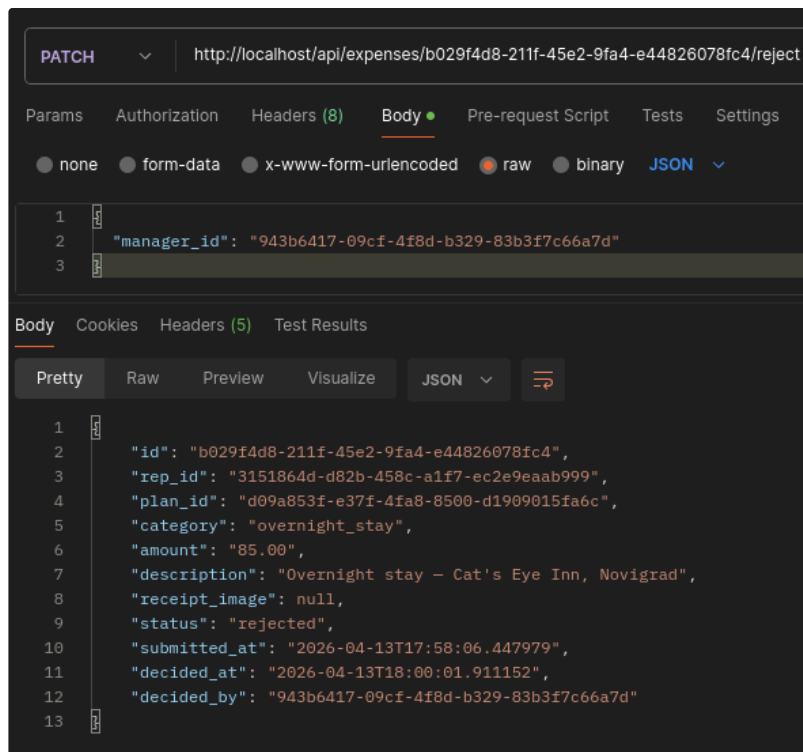
```
1 {
2   "manager_id": "943b6417-09cf-4f8d-b329-83b3f7c66a7d"
3 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize **JSON** ▼

```
1 {
2   "id": "b029f4d8-211f-45e2-9fa4-e44826078fc4",
3   "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
4   "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
5   "category": "overnight_stay",
6   "amount": "85.00",
7   "description": "Overnight stay - Cat's Eye Inn, Novigrad",
8   "receipt_image": null,
9   "status": "pending_approval",
10  "submitted_at": "2026-04-13T17:58:06.447979",
11  "decided_at": null,
12  "decided_by": null
13 }
```

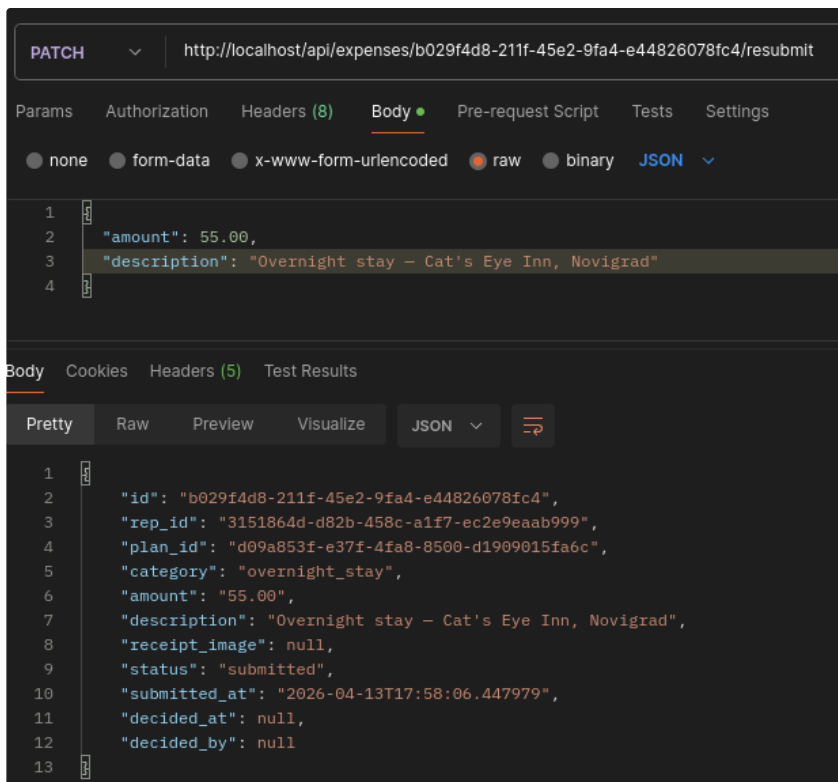
Rejection:



Expense Service Log:

```
2026-04-13 18:00:01 [DEBUG] Database session opened.
2026-04-13 18:00:01 [DEBUG] Projection updated for expense b029f4d8-211f-45e2-9fa4-e44826078fc4.
2026-04-13 18:00:01 [INFO] Expense projection upserted for expense b029f4d8-211f-45e2-9fa4-e44826078fc4.
2026-04-13 18:00:01 [INFO] Published event 'expense.decided' to topic 'expense.decided'.
2026-04-13 18:00:01 [INFO] Expense b029f4d8-211f-45e2-9fa4-e44826078fc4 rejected by manager 943b6417-09cf-4f8d-b329-83b3f7c66a7d.
2026-04-13 18:00:01 [DEBUG] Database session closed.
INFO: 172.18.0.12:57990 - "PATCH /expenses/b029f4d8-211f-45e2-9fa4-e44826078fc4/reject HTTP/1.0" 200 OK
2026-04-13 18:00:01 [DEBUG] Received expense.decided message at offset 1.
2026-04-13 18:00:01 [INFO] expense.decided received: expense_id=b029f4d8-211f-45e2-9fa4-e44826078fc4 decision=rejected.
```

Resubmit:

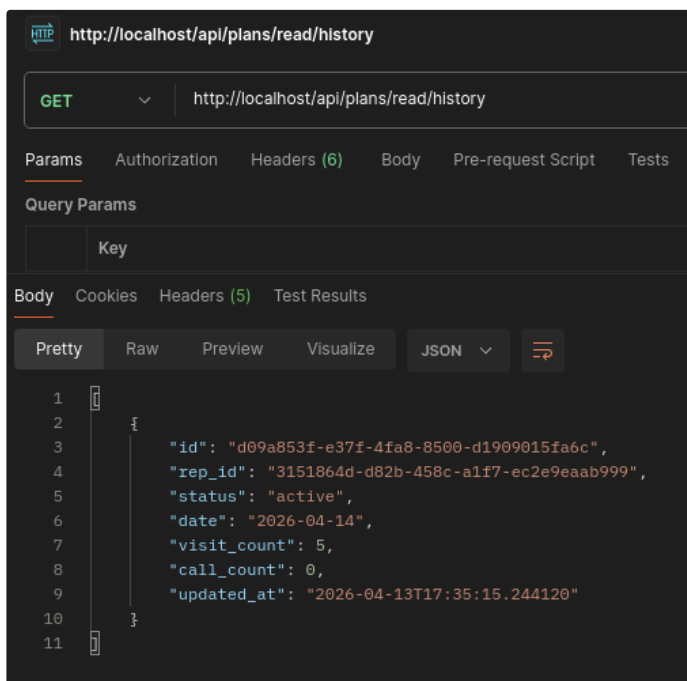


Expense Service Log:

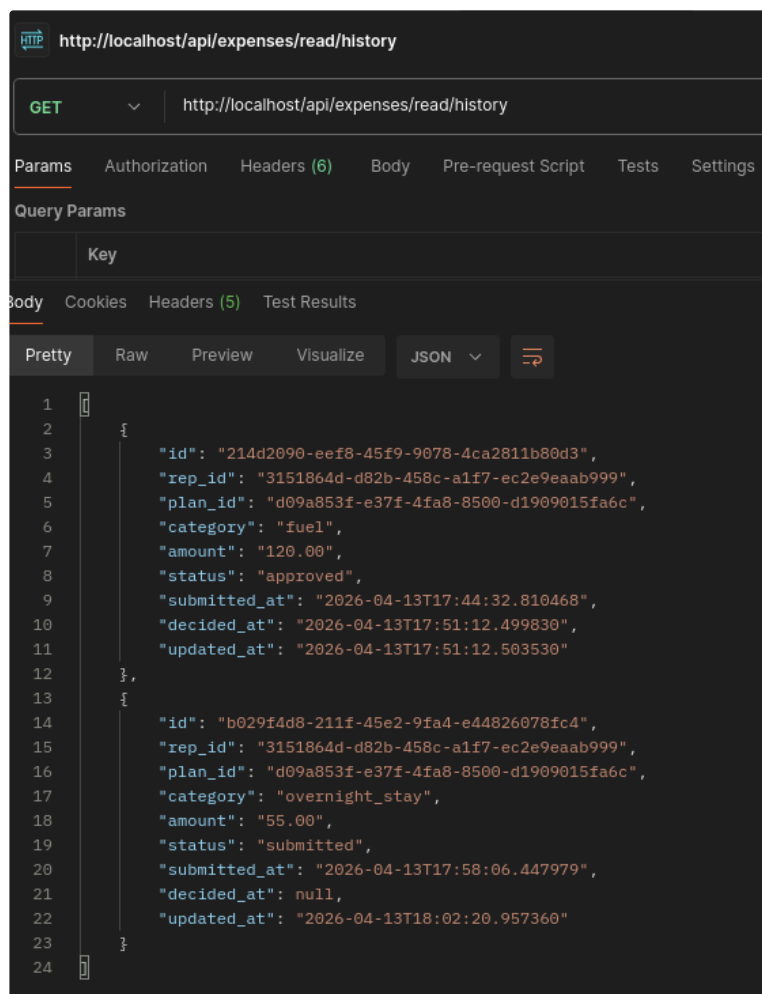
```
2026-04-13 18:02:20 [DEBUG] Database session opened.
2026-04-13 18:02:20 [DEBUG] Projection updated for expense b029f4d8-211f-45e2-9fa4-e44826078fc4.
2026-04-13 18:02:20 [INFO] Expense projection upserted for expense b029f4d8-211f-45e2-9fa4-e44826078fc4.
2026-04-13 18:02:20 [INFO] Expense b029f4d8-211f-45e2-9fa4-e44826078fc4 resubmitted.
2026-04-13 18:02:20 [DEBUG] Database session closed.
INFO: 172.18.0.12:45200 - "PATCH /expenses/b029f4d8-211f-45e2-9fa4-e44826078fc4/resubmit HTTP/1.0" 200 OK
```

CQRS:

Plans History:



Expense Submission History:



```
HTTP http://localhost/api/expenses/read/history
GET http://localhost/api/expenses/read/history
Params Authorization Headers (6) Body Pre-request Script Tests Settings
Query Params
Key
Body Cookies Headers (5) Test Results
Pretty Raw Preview Visualize JSON
1 {
2   {
3     "id": "214d2090-eef8-45f9-9078-4ca2811b80d3",
4     "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
5     "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
6     "category": "fuel",
7     "amount": "120.00",
8     "status": "approved",
9     "submitted_at": "2026-04-13T17:44:32.810468",
10    "decided_at": "2026-04-13T17:51:12.499830",
11    "updated_at": "2026-04-13T17:51:12.503530"
12  },
13  {
14    "id": "b029f4d8-211f-45e2-9fa4-e44826078fc4",
15    "rep_id": "3151864d-d82b-458c-a1f7-ec2e9eaab999",
16    "plan_id": "d09a853f-e37f-4fa8-8500-d1909015fa6c",
17    "category": "overnight_stay",
18    "amount": "55.00",
19    "status": "submitted",
20    "submitted_at": "2026-04-13T17:58:06.447979",
21    "decided_at": null,
22    "updated_at": "2026-04-13T18:02:20.957360"
23  }
24 }
```

8. Reflection & Future Plans

Overall the project was not bad, and I feel I learned more about how to integrate Microservices into real world systems. The biggest shame is not learning this before my Internship in IFM and the 4th Year Project as these concepts would have helped significantly.

For the future, I would like to add proper user log in, authentication and authorization. Along with this, fix the User Interface, I feel I could not get it right for release 1.

For features, I would like to add more sorting to queries, for better GDPR adherence.

9. Use of AI

AI was used throughout the code aspect of this project as per the project spec.

All code was generated using Claude Code CLI.

All prompts were written by myself, Luke Doyle.

To keep prompts consistent across the project, I defined a template at the start and used it for every prompt from scaffolding through to the final service. Rather than restating file paths, import structures, and code standards in every prompt, I described what I wanted in plain terms and used AI to convert that into a properly formatted prompt following the template. This meant every prompt going into Claude Code was structured the same way and I was not repeating myself.

Template:

```
1 <goal of prompt>.  
2  
3 files:  
4 - path/to/file.py  
5 - path/to/other.py  
6  
7 filename.py:  
8 what this file should do.  
9 each responsibility/function on its own line.  
10 short and direct.  
11  
12 imports:  
13 specific import paths or notes about container mount structure.  
14  
15 standards:  
16 follow pep8 at .claude/pep8_python.md.  
17 follow pep257 at .claude/pep257_python_docstring.md.
```

For the standards, I copied the Python Standards at [PEP 8 – Style Guide for Python Code | pe](https://peps.python.org/pep-0008/)
[ps.python.org](https://peps.python.org/pep-0257/) and [PEP 257 – Docstring Conventions | peps.python.org](https://peps.python.org/pep-0257/) into Markdown and had them for quick reference on my local machine.